

ChatGPT generated explanation for the tleap script with light editing by Travis Hughes.

This `tleap` script builds an AMBER topology and coordinate set for a protein–ligand system using **ff19SB** for the protein, **GAFF2** for the ligand, and **OPC** water.

```
source leaprc.gaff2
```

Loads the **GAFF2** force field for small organic molecules.

This defines general AMBER atom types, bonded terms, Lennard-Jones parameters, and other force-field rules needed for nonstandard molecules such as your ligand. GAFF2 is commonly used for drug-like ligands.

```
source leaprc.protein.ff19SB
```

Loads the **ff19SB protein force field**.

This provides parameters for standard amino acids in the protein: atom types, charges, bond/angle/torsion terms, impropers, and residue templates for proteins.

So after this line, `tleap` knows how to interpret standard residues like ALA, GLY, ASP, LYS, etc.

```
source leaprc.water.opc
```

Loads the **OPC water model**.

This defines the OPC water residue template and associated ion/water parameters. OPC is a modern four-site water model, and this line makes the `OPCBOX` solvent box available for solvation.

```
loadamberparams LIG_gaff2.frcmod
```

Loads an AMBER `frcmod` parameter file for the ligand.

The `.frcmod` file contains any ligand parameters that were missing or specially assigned during ligand parameterization. For example, it may include:

```
MASS
BOND
ANGLE
DIHE
IMPROPER
NONBON
```

This file usually comes from Antechamber/Parmchk2. It supplements GAFF2 with ligand-specific parameters that GAFF2 does not already contain.

```
LIG = loadmol2 LIG_gaff2.mol2
```

Loads the ligand residue template from a MOL2 file and assigns it to the variable `LIG`.

The MOL2 file usually contains:

- ligand atom names
- GAFF2 atom types
- partial charges, often AM1-BCC or RESP-style charges
- bond connectivity
- residue name, often `LIG`

This line teaches `tLeap` what the ligand residue is supposed to look like.

Important subtlety: this line **does not place the ligand into the final system by itself** unless `LIG` is later used directly. In your script, the final system uses `component1 = loadpdb ligand_docked.pdb`, not `LIG` directly.

That means `LIG = loadmol2 LIG_gaff2.mol2` is mainly being used as a **template** so that when `ligand_docked.pdb` is loaded, `tLeap` can match the ligand residue in the PDB to the MOL2-defined ligand residue.

For that to work correctly, the residue name and atom names in `ligand_docked.pdb` should match those in `LIG_gaff2.mol2`.

```
component0 = loadpdb abl.pdb
```

Loads the protein structure from `abl.pdb` and stores it in the variable `component0`.

Here, `tLeap` reads the PDB file and tries to assign force-field templates to all residues.

Because you already loaded `leaprc.protein.ff19SB`, standard protein residues should be recognized automatically.

This file is probably the ABL protein structure.

Potential things `tLeap` may warn about here include:

- missing atoms
- unknown residue names
- nonstandard protonation states

- missing terminal caps
- histidine naming issues, such as HIS, HID, HIE, HIP
- missing disulfide bond definitions
- alternate locations in the PDB

```
component1 = loadpdb ligand_docked.pdb
```

Loads the docked ligand coordinates from `ligand_docked.pdb` and stores them in `component1`.

This PDB file likely contains the ligand in its docked pose relative to the protein.

Because a PDB file usually does **not** contain GAFF2 atom types or partial charges, `tleap` must match the residue and atom names in this PDB to the ligand template loaded earlier from:

```
LIG = loadmol2 LIG_gaff2.mol2
```

So this line only works properly if `ligand_docked.pdb` is compatible with the MOL2 template.

For example, if the MOL2 ligand residue is named `LIG`, the PDB should also use residue name `LIG`. Atom names should also match.

```
system = combine {component0 component1 }
```

Combines the protein and ligand into a single molecular system called `system`.

After this line:

- `component0` is the protein
- `component1` is the ligand
- `system` is the combined protein–ligand complex

This combined unit is what will be solvated, ionized, and saved.

The ligand coordinates remain where they were in `ligand_docked.pdb`; `combine` does not perform docking or fitting. It simply merges the loaded components into one AMBER unit.

```
solvateoct system OPCBOX 12.0
```

Solvates the system in a truncated octahedral box of OPC water.

Breaking it down:

```
solvateoct
```

Means use a **truncated octahedron** periodic box rather than a rectangular box.

system

The unit being solvated.

OPCBOX

The pre-equilibrated box of OPC water molecules.

12.0

Adds enough water so that the solute is at least approximately **12 Å** away from the edge of the periodic box.

This creates a solvated protein–ligand complex with a 12 Å solvent buffer.

A truncated octahedron is often more efficient than a rectangular box for roughly globular biomolecules because it uses fewer waters for a similar minimum solute–box distance.

```
addions system Na+ 31
```

Adds **31 sodium ions** to the system.

Na+ is the AMBER sodium ion residue name.

This places 31 positive ions into the solvated system, usually replacing water molecules with sodium ions at favorable electrostatic positions.

Because the number is explicitly given as 31, this line adds exactly 31 sodium ions. It is not automatically calculating the amount needed for a particular salt concentration.

```
addions system Cl- 24
```

Adds **24 chloride ions** to the system.

Cl- is the AMBER chloride ion residue name.

Together, the two ion lines add:

```
31 Na+
24 Cl-
```

That gives a net ionic charge contribution of:

$$31(+1) + 24(-1) = +7$$

So the added ions contribute a total charge of +7.

That likely means the original protein–ligand complex had a net charge of about -7 , and these ions neutralize it. But you should verify this with:

```
charge system
```

or by checking the `tleap` output.

If the complex was not -7 before adding ions, then the final system may not be neutral.

```
saveamberparm system abl_ligand.prmtop abl_ligand.inpcrd
```

Writes the final AMBER topology and coordinate files.

This creates two files:

```
abl_ligand.prmtop
```

The AMBER topology file. It contains the force-field information: atom types, charges, masses, bonds, angles, dihedrals, nonbonded parameters, residue definitions, box information, etc.

```
abl_ligand.inpcrd
```

The AMBER coordinate file. It contains the atomic coordinates and periodic box dimensions.

These two files are the main inputs for AMBER simulations, such as minimization, heating, equilibration, and production MD.

```
quit
```

Exits `tleap`.

Overall workflow

Your script does the following:

1. Loads GAFF2 for the ligand.
2. Loads ff19SB for the protein.
3. Loads OPC water parameters.
4. Loads ligand-specific missing parameters from `LIG_gaff2.frcmod`.
5. Loads the ligand MOL2 file as a residue template.
6. Loads the protein PDB.

7. Loads the docked ligand PDB.
 8. Combines protein and ligand.
 9. Solvates the complex in a truncated octahedral OPC water box with a 12 Å buffer.
 10. Adds 31 sodium and 24 chloride ions.
 11. Saves AMBER topology and coordinate files.
-

One important thing to check

This part is potentially fragile:

```
LIG = loadmol2 LIG_gaff2.mol2
component1 = loadpdb ligand_docked.pdb
```

This works only if `ligand_docked.pdb` matches the ligand template loaded from `LIG_gaff2.mol2`.

In practice, check that:

- the ligand residue name is the same in both files;
- the atom names are the same in both files;
- the ligand has the same protonation state;
- the ligand has the same bond order/connectivity;
- `tLeap` does not report unknown atoms or missing parameters.

A safer diagnostic version of the script would include:

```
check component0
check component1
check system
charge system
savepdb system ab1_ligand_solvated.pdb
```

before `saveamberparm`, so you can inspect whether the complex was built correctly.